

Documento Técnico

Jogo de Exploração RPG

Versão: 1.0

Linguagem: HTML5 + CSS3 + JavaScript Vanilla

Arquitetura: Aplicação Web Cliente (Client Side)

1. Objetivo

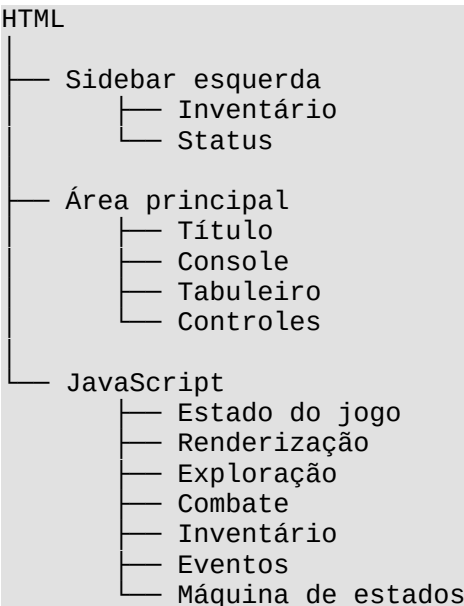
O sistema consiste em um jogo RPG de exploração em grade (grid) desenvolvido inteiramente em HTML, CSS e JavaScript puro, sem dependência de frameworks externos.

O jogador explora um mapa desconhecido, encontra criaturas e baús, coleta equipamentos, participa de combates por turno e evolui até concluir toda a exploração do mapa.

O projeto utiliza uma arquitetura baseada em máquina de estados (State Machine), controlando todas as ações do jogador através de estados definidos.

2. Estrutura Geral

O sistema é dividido em três camadas principais:



3. Interface Gráfica

A interface possui resolução fixa de:

850 x 850 pixels

dividida em:

Sidebar Esquerda

Contém:

- Inventário
 - HP
 - ATK
 - DEF
 - Informações do inimigo durante combate
-

Área Principal

Contém:

- Nome do jogo
 - Console de mensagens
 - Tabuleiro 8x8
 - Controles
-

4. Sistema Visual

O projeto utiliza uma identidade visual retrô inspirada em RPGs dos anos 90.

Principais características:

- Tema escuro
- Fonte pixelada
- Scanlines simulando CRT
- Barra de HP
- Gradientes
- Texturas de grama
- Marcadores por emoji

Exemplos:

- ▣ Jogador
- ▣ Inimigo

5. Sistema de Mapa

O mapa possui:

8 linhas
8 colunas

Total:

64 células

Cada célula pode possuir:

0 = segura
1 = criatura
2 = baú

As matrizes utilizadas são:

logicMatrix
visualMatrix
entityMatrix

6. Matrizes

logicMatrix

Responsável pela lógica do jogo.

0 -> segura
1 -> criatura
2 -> baú

visualMatrix

Responsável pela aparência.

Estados:

unexplored
explored
current

entityMatrix

Armazena objetos permanentes.

```
null  
goblin  
chest
```

7. Sistema de Coordenadas

O mapa utiliza notação semelhante ao xadrez.

Exemplo:

```
A1  
B4  
C7  
H8
```

A posição inicial do jogador é:

```
A1
```

Internamente:

```
row = 7  
col = 0
```

8. Distribuição Procedural

Durante a geração do mapa:

```
70% células vazias  
15% criaturas  
15% baús
```

A posição inicial é sempre segura.

9. Jogador

A estrutura do jogador contém:

```
HP  
HP máximo
```

ATK

DEF

Amuleto da Vida

Estado inicial:

HP = 100

ATK = 2

DEF = 0

10. Inventário

Os itens ficam armazenados em um vetor:

inventory[]

Cada item possui:

nome

tipo

raridade

efeito

atributos

11. Raridade

O sistema possui três categorias:

Comum

60%

Exemplo:

- Espada de Madeira
 - Armadura Velha
 - Escudo
 - Pinga
-

Médio

25%

Exemplo:

- Espada Velha
 - Espada Amaldiçoada
 - Armadura Amaldiçoada
 - Bandagens
-

Raro

15%

Exemplo:

- Espada do Anjo
 - Armadura do Anjo
 - Escudo Angelical
 - Amuleto da Vida
-

12. Sistema de Exploração

Movimentação:

↑
↓
←
→

Caso a casa seja segura:

- jogador movimenta
- área é revelada

Caso haja evento:

o jogo solicita confirmação.

Prosseguir?

[Y]

[N]

13. Sistema de Mimicry

Existe uma mecânica de inversão aleatória.

Ao entrar em uma célula perigosa:

```
50%
continua igual
50%
troca:
criatura ↔ baú
```

Isso adiciona imprevisibilidade à exploração.

14. Sistema de Combate

O combate é baseado em turnos.

Fluxo:

```
Jogador
↓
Inimigo
↓
Jogador
↓
Inimigo
```

Cada turno o jogador escolhe:

```
Atacar
ou
Defender
```

15. Ataque

O dano básico é:

```
ATK do jogador
```

Caso o inimigo esteja defendendo:

```
dano =
```

ATK

-

DEF inimigo

16. Defesa

Quando o jogador defende:

recebe redução de dano

usa DEF

Após três defesas consecutivas:

fuga automática

17. Sistema de Fuga

Antes do combate começar:

chance =

1 -

(casas exploradas / total)

Quanto menos explorado o mapa, maior a chance de fugir.

18. Inimigos

Os monstros são escolhidos aleatoriamente:

- Goblin
- Bugbear
- Owlbear
- Displacer Beast
- Kobold
- Hobgoblin
- Gnoll
- Troll
- Ogre

- Minotaur
- Basilisk
- Manticore

Os atributos são gerados próximos aos atributos atuais do jogador.

19. Sistema de Baús

Ao abrir um baú:

- gera item aleatório
 - adiciona ao inventário
 - aplica efeitos permanentes ou consumíveis
-

20. Consumíveis

Pinga

HP +10

DEF -2

Bandagem

HP +15

21. Equipamentos

Equipamentos são aplicados imediatamente ao jogador.

Podem alterar:

ATK

DEF

HP máximo

Alguns possuem maldição.

22. Amuleto da Vida

Item raro passivo.

Após vencer combate:

```
+20 HP
```

até o HP máximo.

23. Máquina de Estados

O jogo inteiro é controlado pela variável:

```
state
```

Estados disponíveis:

```
explore
confirm_move
combat_init
combat_turn
use_item
victory
gameover
new_game_plus
```

Cada estado aceita apenas determinadas entradas do teclado.

24. Controles

```
Setas
```

```
Movimento
```

```
Y
```

```
Confirmar / Atacar
```

```
N
```

```
Cancelar / Defender
```

```
Espaço
```

```
Inventário
```

```
↑↓
```

Selecionar item

Enter

Usar item

25. Condição de Vitória

Quando todas as células forem exploradas:

64 células visitadas

o jogador vence.

O jogo oferece continuar para uma nova floresta.

26. New Game Plus

Ao aceitar:

- novo mapa
 - exploração reiniciada
 - geração incrementada
 - jogador mantém progresso de atributos e inventário existente na implementação
-

27. Tecnologias Utilizadas

- HTML5
- CSS3
- JavaScript ES6
- CSS Variables
- CSS Grid
- Flexbox
- DOM API
- Eventos de teclado
- Programação procedural
- Máquina de estados
- Geração pseudoaleatória (`Math.random()`)

28. Considerações Técnicas

O projeto apresenta uma arquitetura simples, porém bem organizada, utilizando separação lógica entre renderização, regras de negócio, combate, exploração e inventário. A utilização de múltiplas matrizes para lógica, visualização e entidades desacopla a representação gráfica da lógica interna do jogo, facilitando futuras expansões como novos inimigos, NPCs, missões ou mapas maiores. A máquina de estados centraliza o fluxo de execução, reduz conflitos entre entradas do teclado e torna o comportamento do jogo previsível e extensível.